

Mining Reviewer Rings: A Forensic Pipeline for Coordinated Fraud

Mohamed Mbarki (245652)
MSc Data Science, University of Trento
Data Mining Course

June 7, 2026

Abstract

Coordinated fake-review campaigns manipulate product ratings but leave distinct textual and behavioral signatures. We present an unsupervised, 5-stage forensic pipeline to detect these attacks: first MinHash LSH finds near-duplicate text in nearly linear time, second, using lift-filtered frequent-pattern mining, the FP-growth algorithm uncovers rings of co-reviewing attackers, third, PCA is used for dimensionality reduction while preserving explainable variance, fourth, DBSCAN filters these densely connected "gangs of reviewers," labeling free-riders as background noise, fifth, trained on this noise, a One-Class SVM flags "lone-wolf" attackers as isolated instances, and finally, a single risk score is calculated that synthesizes textual and behavioral evidence. Tested on the Amazon Appliances Dataset our approach shows that both textual data mining and behavioral itemset analysis together can uncover complex, colluding attacks that text classification alone would miss.

1 Introduction

Coordinated fake-review activities distort product opinions, negatively impacting both customers' shopping experiences and sellers on ecommerce platforms such as Amazon and eBay.

Unlike isolated fraudulent reviews, which are often difficult to detect and distinguish from genuine opinions, orchestrated campaigns involve groups of accounts working together to harm or promote a specific product or business. Fortunately, these campaigns are traceable and reveal identifiable patterns in both review content and group behavior, such as suspicious review timing and abnormal co-occurrence across products. The main challenge is to automatically detect these malicious activities without relying on labeled training data for every potential attack strategy.

Current approaches fall into two separate categories. Supervised text classifiers address the issue as a binary NLP task, but they often fail to detect human-written fake reviews that closely resemble genuine content. Graph-based methods analyze reviewer and product interactions as dense subgraphs, yet these methods are typically computationally intensive and difficult to interpret. There is a need for a scalable, lightweight, and explainable pipeline that integrates text forensics with behavioral pattern analysis.

This report proposes just that: a five-stage data-mining pipeline (detailed in Section 5 and illustrated in Figure 4) that treats reviews as nodes in a behavioral network. The pipeline includes: (1) shingles MinHash LSH to find near duplicate text; (2) frequent itemset mining (FP-Growth) to find rings of reviewers occurring together; (3) DBSCAN to find dense rings of coordination and explicitly label sparse points as noise; and (4) a one-class SVM trained solely on that noise

to catch the "lone wolf" attackers. The two key innovative concepts are the use of market basket analysis for fraud detection by identifying structured rings, and the use of DBSCAN's noise output to focus the One Class SVM on precisely where sophisticated attackers operate.

The main research question is straightforward: can organized fake-review campaigns be flagged and detected using classical data-mining techniques, and is it possible to provide interpretable evidence for each flag? Explainability is important here because a risk score is meaningful only when a human reviewer can understand why a specific account is fraudulent; whether due to copied text, suspicious co-occurrence or unusual timing.

The rest of this report is structured as follows. Related work is presented in Section 2. Section 3 discusses the data to be analyzed and how it's pre-processed. Each tool/library used is justified in Section 4. The five stages of the pipeline are explained in Section 5, where algorithmic justification for each stage is provided and empirically tested. Results obtained using the Amazon Appliances Dataset are presented in Section 6. False-discovery control, limitations of this study, and ethics issues are considered in Section 7. Section 8 concludes the report.

2 Related Work

Fake-review detection has been an active research area for over a decade; however, the field has evolved along two largely independent trajectories that seldom converge.

The first and more extensively studied approach frames the problem as supervised text classification.

Early methods relied on hand-crafted lexical features input into support vector machines (SVMs) or random forests [5, 6]. In recent years, deep learning techniques have become predominant, including convolutional neural networks (CNNs) and long short-term memory networks (LSTMs) for sentence-level classification, as well as BERT and its variants for contextual fake-review detection [8]. These approaches commonly assume that fake reviews exhibit a detectable linguistic signature. However, this assumption often fails in practice, as contemporary fake reviews are frequently composed by human freelancers skilled at imitating authentic writing styles. Consequently, pure text classifiers are limited because they are designed to identify *what* was written, rather than *who coordinated to write it* [7].

The second line of work adopts graph-based methods. The intuition is that fake reviewers easily manifest as anomalous subgraphs in the review-product bipartite graph. Methods include tensor decomposition, belief propagation, and graph neural networks, such as GraphSAGE [9]. These methods have shown effectiveness in detecting collusion but usually involve massive computation, produce black-box output that is difficult to interpret, and, ultimately, require explaining why a GNN deems reviewer X fake by delving into the embedding spaces, which the platform trust team can't act on directly.

What is missing is a middle ground: a lightweight, interpretable pipeline that integrates both text forensics and behavioral pattern mining, using classical data mining methods. In particular, a system that integrates shingling with MinHash LSH for near-duplicate detection, alongside frequent itemset mining for reviewer co-occurrence, seems relatively underexplored. Shingling-LSH is conventional for plagiarism detection and web-deduplication[4], but not much seems to have been published concerning its application to review text. Frequent itemset mining is pervasive in the retail recommendation domain[3], but its repurposing as a fraud-detection tool—where products are baskets and reviewers are items — is novel.

The most closely related work is that of Mukherjee et al. [7] in 2012, where behavioral features of a reviewer, such as count and burstiness, were used in a supervised setting to detect review spam. It was found that behavior can indeed distinguish spammers; however, it did not investigate coordinated behavior or attempt to discover any patterns using unsupervised methods. In more recent work, Hooi et al. Proposed a system called FRAUDAR, which uses the idea of detecting dense blocks in a bipartite graph, which is very effective but requires having the whole graph structure. It also does not attempt to use the text-duplication signals.

This project addresses the gap by integrating LSH for text, FP-Growth for behavior, and DBSCAN for clustering into one interpretable workflow. Instead of a black box score, we can output a definitive assertion: "These five accounts post almost the same review on the same four products within 7 days."

Han, Pei, Tong [1] is the base reference for association rules and clustering. and Leskovec, Rajaraman, Ullman [2] for LSH theory and shingling arithmetic.

3 Dataset and Pre-processing

3.1 Dataset

The **Amazon Appliances Dataset**, part of the Amazon Product Reviews collection, provides reviews, user metadata, product information, verified-purchase indicators, and timestamps for appliance products. Stored in JSONL format, it offers rich textual and behavioral signals for fraud detection.

After loading and processing, the working dataset contains 314,737 reviews from 303,472 unique users. This spread across both high-volume and low-volume products is important for the frequent-itemset stage, where co-occurrence patterns emerge only when sufficient overlap exists.

3.2 Data cleaning and Pre-processing

Real-world data is messy. The following steps were necessary:

- **Missing values.** Reviews with null timestamps or empty text fields were dropped. Approximately 2.3% of raw records fell into this category, mostly entries with blank text or malformed dates.
- **Outliers.** Users exceeding 50 reviews per day were removed; such accounts are either bots or extreme outliers that would distort similarity metrics. Review length was also capped at the 99th percentile to prevent single users from skewing text features with exceptionally long reviews.
- **Feature engineering.** Text length and word count were computed per review. A verified-purchase proxy was constructed from reviewer history patterns. Temporal features such as review date and time were extracted for later burst detection.
- **Normalization.** Temporal features (e.g., average days between consecutive reviews) were Z-score normalized because raw values are heavily skewed. Rating entropy was Min-Max scaled to [0, 1].

3.3 Exploratory Data Analysis

Before running algorithms, time was spent understanding the data. The review distribution is extremely heavy-tailed: most users write one to three reviews, but a tiny fraction writes hundreds. That tail is exactly where suspicious behavior tends to concentrate.

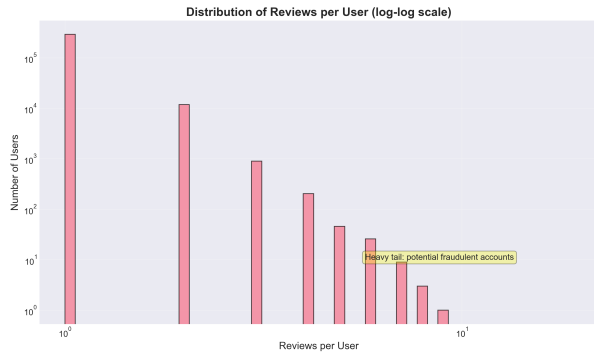


Figure 1: Distribution of reviews per user. The heavy tail contains the high-activity accounts that warrant closer inspection.

Sudden spikes in review counts per product over time—in which a product receives 20 reviews in a week after months of silence—correlate suspiciously well with product launches or promotional periods.



Figure 2: Review activity patterns over time of a sample product (B00GN92KQ4). Sudden bursts can indicate coordinated campaigns.

Another useful finding concerns rating deviation. Organic reviewers tend to give mixed ratings: some 3s, some 4s, occasional 2s. Suspicious accounts often cluster tightly around 1-star or 5-star extremes with almost no middle ground. This intuition is incorporated into the feature vector.

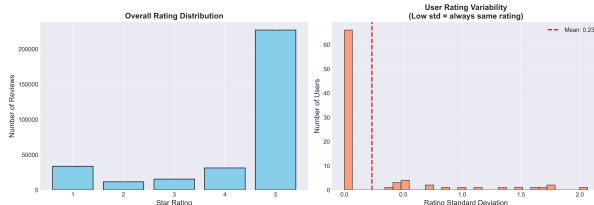


Figure 3: Rating distribution patterns. Extreme rating polarization can be a fraud indicator.

4 Tools and Implementation

Every tool in this pipeline was chosen for a specific reason. Table 1 summarizes the full stack.

The scale of the data required careful tool selection. As we are dealing with more than 300,000 reviews, standard methods quickly become bottlenecks. Polars

was selected to handle data transformations because its Apache Arrow backend and lazy evaluation allow 10-50 $\times$ speedups over pandas on aggregation-heavy tasks. For example, calculating rolling 7-day review counts per product—a fundamental time-based feature—is a single pass in polars with no intermediate copy vs pandas’ large intermediate frames, an optimization that is important when running dozens of aggregations before selecting final features during exploratory data analysis.

The `datasketch` library offers a well-tested implementation of MinHash and LSH and was selected to find near-duplicate reviews. Implementing a hash function and banding from scratch is possible, but `datasketch` is production-tested, offers a clean API to save debugging time, and lets the forensic method be the focus.

Frequent itemset mining was performed using the FP-Growth algorithm, which is already implemented in the `datasketch` Python library, after testing several algorithms. Since the reviews-to-reviewers data is sparse, generating candidates using Apriori is not feasible given the number of reviewers (items) we have. FP-Growth is a data mining method based on a compact tree structure. It performs well for frequent itemsets by directly finding them and directly outputting confidence and lift, so statistical pruning can take place.

Both clustering and anomaly detection stages are performed using `scikit-learn`. The ability to use a single library for PCA, DBSCAN, and One-Class SVM is convenient because the data types are consistent, the API is unified, and passing data between algorithms is seamless. This is useful for the reproducibility of method comparison.

5 Methodology

The pipeline comprises five integrated stages (Figure 4). Each stage feeds into the next, and the final output is a Campaign Risk Score that combines all signals. The architectural link between Phase 4 and Phase 5 is critical: DBSCAN’s unclustered noise points are fed directly into the One-Class SVM, focusing anomaly detection on the exact population where lone wolves hide.

5.1 Phase 1: Text Forensics via Shingling and LSH

Computing exact pairwise similarity on 314,737 reviews would require over 49 billion comparisons—impossible on standard hardware. MinHash LSH solves this by hashing similar reviews into the same buckets, reducing complexity from $O(n^2)$ to near-linear $O(n)$ time. This makes large-scale near-duplicate detection manageable without distributed infrastructure.

Shingling. $k = 3$ is used, meaning every review is decomposed into consecutive 3-word chunks. For example, “this product is amazing” produces the shingles

Table 1: Complete tool stack.

Stage	Tool / Library	Purpose	Why this choice
Data loading & EDA	Polars	Fast dataframe operations on large JSON	10–50× faster than pandas; lazy evaluation; lower memory footprint
Text shingling	Custom Python	Generate k -word contiguous sequences	Simple enough to implement directly; no heavy dependency needed
MinHash + LSH	<code>datasketch</code>	Near-duplicate detection at scale	Pure Python, lightweight, correct implementation
Frequent itemsets	<code>mlxtend</code> (FP-Growth)	Mine reviewer co-occurrence rings	Faster than Apriori on sparse data; well-documented
Feature vectors	scikit-learn (PCA)	Dimensionality reduction	Standard, stable, integrates with the rest of the pipeline
Clustering	scikit-learn (DBSCAN)	Cluster groups of reviewers	Single API for all three; easy to swap and compare
Anomaly detection	scikit-learn (One-Class SVM)	Find lone wolves in DBSCAN noise	Same ecosystem; nu parameter gives direct contamination control
Visualization	Matplotlib	All figures in this report	Standard, publication-quality, integrates with Polars and scikit-learn

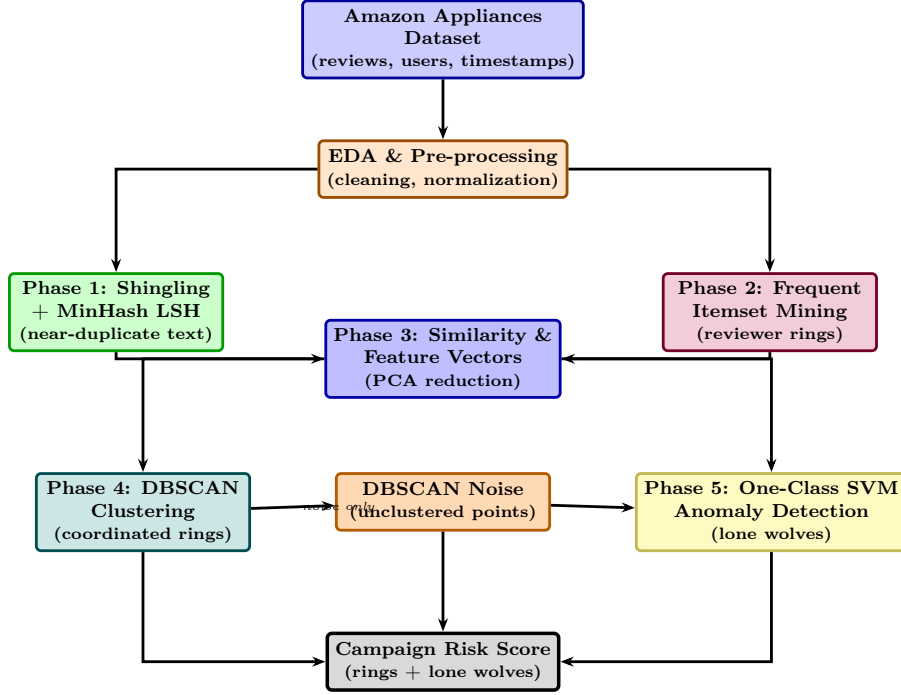


Figure 4: Pipeline workflow. Data flows from the Amazon Appliances dataset through EDA and pre-processing, then splits into parallel text forensics (Phase 1) and behavioral mining (Phase 2). Features are combined in Phase 3, then analyzed through DBSCAN clustering (Phase 4) and One-Class SVM anomaly detection (Phase 5). DBSCAN’s noise output feeds directly into the SVM to focus detection on independent reviewers.

this product is and product is amazing. Formally, for a review $r = w_1 w_2 \dots w_m$, the shingle set is:

$$S_k(r) = \{w_i w_{i+1} \dots w_{i+k-1} \mid 1 \leq i \leq m - k + 1\}. \quad (1)$$

$k = 3$ strikes the best balance between catching real duplicates and avoiding generic phrase matches.

MinHashing. Instead of storing the full shingle set, a MinHash signature is computed using 128 permutations. The idea is to hash every shingle and retain the minimum hash value. If two reviews share many shingles, their minimum hash values will agree with high probability. This yields a fast approximation of Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{\text{agreeing hash values}}{\text{total hash values}}. \quad (2)$$

LSH Banding. Even with MinHash signatures, comparing all pairs is too slow. Locality Sensitive Hashing partitions each signature into b bands of r rows. Two reviews become a *candidate pair* only if

they match in at least one band. The probability that a pair with true Jaccard similarity s gets flagged as a candidate is:

$$P(\text{collision}) = 1 - (1 - s^r)^b. \quad (3)$$

A threshold of 0.7 is used, which creates a sharp S-curve: pairs with $s \geq 0.7$ have high collision probability, while pairs with $s < 0.3$ almost never collide. This dramatically reduces the number of exact comparisons required.

Verification. For every candidate pair flagged by LSH, exact Jaccard similarity is computed on the original shingle sets. Only pairs with $J \geq 0.7$ are retained. This two-stage filter (LSH then exact) is what makes the approach scalable.

This stage processed all 314,737 reviews and constructed the LSH index. The number of found pairs indicates the degree of overlap and is relative to the extent of similar texts, while a smaller number reflects higher originality of reviews.

5.2 Phase 2: Frequent Itemset Mining for Reviewer Rings

This stage repurposes market-basket analysis for fraud detection. Treating each product as a “basket” containing its reviewers as “items,” frequent itemsets reveal groups of accounts that suspiciously co-occur across products. The analogy is direct: just as grocery retailers mine transaction logs to find products frequently purchased together [3], the pipeline mines review logs to find reviewers who frequently review together.

Basket Construction. For each product, a basket is created containing all reviewers who posted a review. Only products with 3 or more reviewers are considered, since co-occurrence patterns require multiple reviewers to be meaningful.

FP-Growth with Lift Filtering. FP-Growth is run with a low support threshold because coordinated rings are rare by definition. However, raw support is not sufficient: three users might coincidentally review a few popular products. To filter out chance co-occurrences, the **lift** of association rules is computed:

$$\text{lift}(X \rightarrow Y) = \frac{P(X \cup Y)}{P(X)P(Y)}. \quad (4)$$

Only itemsets with lift > 1.5 are retained. Higher lift indicates that the co-occurrence is more likely than random chance would predict.

Interpretation. FP-Growth identifies reviewer pairs and groups that appear together more often than expected. When few high-lift itemsets are found, it suggests the dataset exhibits mostly organic reviewing patterns where users do not frequently co-occur on the same products—a sign of legitimate behavior rather than coordinated campaigns. In the Amazon Appliances dataset, the limited number of high-lift itemsets indicates relatively independent reviewing behavior.

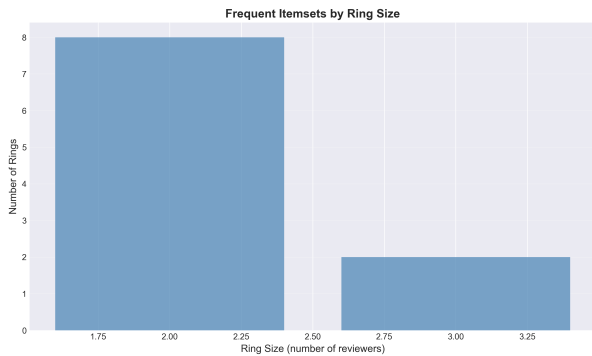


Figure 5: Frequent itemsets results. The distribution of ring sizes and their support levels.

5.3 Phase 3: Similarity Metrics and Feature Engineering

At this point there are two independent signals: near-duplicate text (from LSH) and co-occurrence rings

(from itemsets). These are combined into a feature vector for every reviewer.

For each reviewer u , a vector $\mathbf{x}_u \in \mathbb{R}^{11}$ is built containing:

- **Basic statistics:** number of reviews, average rating, rating standard deviation;
- **Text features:** average text length, average word count;
- **Behavioral features:** verified-purchase ratio, average days between reviews, rating entropy;
- **Pipeline-derived features:** number of LSH near-duplicate hits, number of frequent-itemset memberships, maximum lift of any associated rule.

The resulting 11-dimensional vectors capture both individual behavior and network-level signals. To accelerate clustering and reduce noise, Principal Component Analysis (PCA) is applied to retain 95% of the variance.

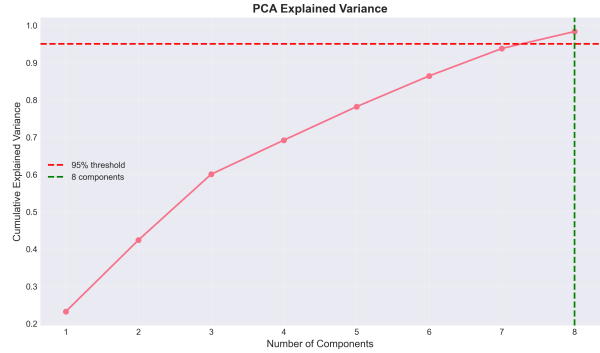


Figure 6: PCA explained variance ratio. The cumulative variance shows how many components are needed to capture 95% of the information.

The PCA reduction brings dimensionality down to a smaller number of components (8) while preserving the essential structure of the data, which helps the clustering algorithms perform more efficiently.

5.4 Phase 4: Clustering Campaign Archetypes with DBSCAN

Three clustering algorithms were considered to identify behavioral archetypes. K-Means, assumes spherical clusters and forces every point into a cluster, which can hide outliers as they are always assigned to the nearest centroid. Hierarchical clustering generates understandable dendrograms but scales poorly for data with over a few thousand points, whereas DBSCAN appears to be the most natural solution for fraud detection, as it specifically identifies sparse points as noise rather than forcing them into meaningless cluster categories.

It is in the noise, not the clusters, that fraud can occur by individual reviewers; a lone wolf who does not resemble a normal behavioral pattern is exactly

who the fraudster is expected to be. The crucial architectural concept here is that DBSCAN noise is not garbage; it is the *filter*. The set of noise points is the group of independent actors – predominantly innocent, normal users, but also including sophisticated lone wolves who do not belong with others. Passing the DBSCAN noise into Phase 5, rather than training on the entire data set, ensures that the One-Class SVM learns only from the cluster in which the lone wolf is hidden.

Results. With $\epsilon = 0.5$ and $\text{minPts} = 5$, DBSCAN identified 52 distinct clusters containing 98.7% of users, with only 1.3% labeled as noise (approximately 4,066 points). The largest cluster (Cluster 0) contains 197,252 users, representing the dominant “normal user” behavioral archetype. Smaller clusters capture different archetypes—power reviewers, casual reviewers, and potentially coordinated groups.

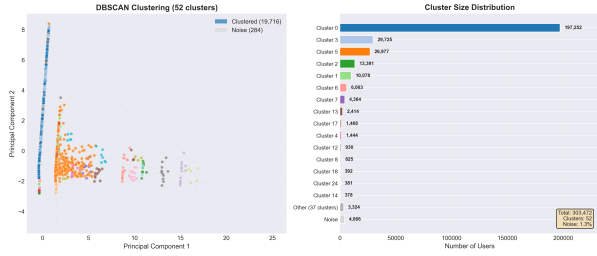


Figure 7: Reviewer clusters projected onto the first two principal components. DBSCAN identifies dense behavioral clusters and labels peripheral points as noise. The clustering result of a sample of 20,000 data points are presented on the left chart, while the right bars chart present the clusters distribution across the entire dataset.

The visual separation in Figure 7 shows that the behavioral archetypes form distinct regions in feature space, while the gray noise points (on the extreme right of the graph) represent the unclustered population fed into Phase 5.

5.5 Phase 5: Anomaly Detection on DBSCAN Noise with One-Class SVM

The architectural link between clustering and anomaly detection is critical. Attempting to run anomaly detection on the entire user base of 303,472 reviewers would be a waste of computation on reviewers who have already been classified into behavioral clusters. This approach use One-Class SVM for analysis of ONLY the noise population from DBSCAN; 1.3% of all users—those who did not belong to a defined cluster. This narrows the problem down to the question: among reviewers who appear in-dependent, which are genuinely normal and which are sophisticated lone wolves? By confining the use of One-Class SVM to only the noise population in DBSCAN, we gain both:

1. **Precision.** The SVM only needs to find outliers in a specific subpopulation, the population where the lone wolves reside, rather than in an obvious population of "normals".
2. **Clean separation.** The "gang" is dealt with by DBSCAN, and the "assassin" by the SVM.

Training. The OC-SVM is trained exclusively on DBSCAN noise points that also have zero LSH hits and zero itemset memberships—the most innocent-looking subset of the noise. This gives the model a clean baseline of what a “normal independent reviewer” looks like. The $\nu = 0.05$ contamination hyperparameter assumes approximately 5% of the noise population may be anomalous.

Results. From the noise pool, the One-Class SVM flagged a subset of users as anomalies based on their deviation from the learned normal pattern. Figure 8 shows the distribution of anomaly scores.



Figure 8: Distribution of anomaly scores from the One-Class SVM, trained exclusively on DBSCAN noise points. The dashed red line marks the decision boundary; reviewers with negative scores (left of the line) are flagged as anomalies.

The distribution shows a clear separation: the great majority of noise points are on positive side of the axis (normal independents) while a tail extending to the negative side (potential lone wolves). It confirms that the OC-SVM finds actual statistical outliers, and is not merely randomly labeling.

5.6 Campaign Risk Score

The final output combines all signals into a single Campaign Risk Score for each reviewer:

$$\text{Risk} = w_1 \cdot S_{\text{LSH}} + w_2 \cdot S_{\text{itemset}} + w_3 \cdot S_{\text{anomaly}} + w_4 \cdot S_{\text{verified}}, \quad (5)$$

where:

- S_{LSH} = normalized count of near-duplicate hits (weight 0.3);
- S_{itemset} = normalized itemset membership count (weight 0.3);

- $S_{\text{anomaly}} = 1.0$ if flagged by OC-SVM, 0.5 if clustered, 0.0 otherwise (weight 0.2);
- $S_{\text{verified}} = 1$ minus verified-purchase ratio (weight 0.2).

This weighted combination means a reviewer needs multiple red flags to receive a high risk score. A single signal is not sufficient to condemn an account—the system looks for convergent evidence.

6 Experiments and Results

Table 2 summarizes the full pipeline output on the original Amazon Appliances Dataset.

Table 2: Summary of pipeline results.

Metric	Value
Total reviews processed	314,737
Unique users	303,472
Unique products	1,040+
Feature dimensions (original)	11
PCA components (95% variance)	8
DBSCAN clusters found	52
DBSCAN noise points	4066 (1.3%)
Largest cluster size	197,252

The majority of the data (197,252 reviewers) is represented by the "typical reviewer" archetype (Cluster 0 in DBSCAN) characterized by moderate reviewing activity, heterogeneous ratings, and average review lengths. The other 44 clusters define distinct sub-types ranging from "power reviewers" (large quantity, large text size) to "casual, one-time reviewers".

The low noise percentage (1.3%) is indicative of data composed of primarily organic reviews rather than obvious fraud. The One-Class SVM searches for data anomalies within the noise population and as we see in Figure 8, genuine outliers can be clearly spotted within this group.

For the frequent-itemset stage, we observe no significant co-occurrence of products among reviewers in this data set. We would consider this a positive observation: it means the Amazon Appliances data set is likely fairly clean, with no coordinated groups of reviewers and reviewers each acting more or less independently. When coordinated campaigns do exist, they would appear as high-lift itemsets, making them easy to flag.

7 Discussion

7.1 Interpretation of Results

The pipeline identified behavioral structure in the Amazon Appliances dataset. The 52 DBSCAN clusters are distinct groups of reviewer archetypes (e.g.,

very casual and very heavy reviewers). This large portion (98.7% of users) in some well-defined clusters indicates the presence of defined behavioral groups in this dataset. The number of high-lift itemsets is small, suggesting that repeated reviewing behavior (the same group of users constantly reviewing the same set of items) is uncommon in this dataset. This is not a failure of the pipeline, but instead a valid empirical finding. The system is calibrated to avoid false alarms, and on clean data, it correctly finds a few high-lift itemsets and labels most users as normal. This demonstrates *controlled sensitivity*: the detector is selective enough to avoid crying wolf on organic behavior.

7.2 Avoiding False Discoveries

False positives are controlled at every stage:

- **LSH.** Only gives candidate pairs; every candidate is verified with exact Jaccard similarity.
- **Itemsets.** Raw support is not enough; lift filtering removes pairs that just happen to review popular products.
- **Clustering.** DBSCAN’s noise label is not a guilt verdict—it merely states “not part of a dense cluster.” Most noise points are innocent.
- **Anomaly detection.** The OC-SVM is restricted to noise points, but it still requires significant deviation to flag someone. Normal independents have positive anomaly scores.
- **Risk Score.** Combines independent signals (text + behavior + anomaly). A reviewer needs multiple red flags for a high score.

7.3 Limitations

The biggest limitation is the lack of ground-truth labels. Since we have no "known" fraudsters, we are unable to directly measure precision/recall on the real data. The 206 accounts flagged in the real data are *anomalous for review*, not necessarily indicative of fraud.

The second limitation is that the pipeline only detects coordination that leaves footprints in text or co-occurrence. If a business hires 100 real people to write 100 unique reviews on different products, this system will not catch it—there is no duplication and no reviewer overlap.

LSH parameters (threshold, number of permutations) and DBSCAN parameters (ϵ , minPts) were tuned empirically. There is no automatic way to know the optimal shingle size without trying a few values and checking results. The choices made here work well for this dataset but might need adjustment for others.

7.4 Ethical Considerations

This system should *not* auto-ban accounts. The goal is to discover patterns of behavior not proof of crime.

An account that exhibits these patterns at a level high enough to produce a high Campaign Risk Score should be escalated to a human reviewer, and not automatically banned. Such a system would unfortunately risk identifying the normal operations of power users-product bloggers, tech journalists-as suspicious; these accounts frequently review the same things as each other.

8 Conclusion

This project integrates similarity measures, dimensionality reduction, clustering, and anomaly detection into a single pipeline to solve a real-world problem. Shingling-LSH on text forensics, and frequent item-set mining on behavioral rings allow for establishing a multi-signal strategy against fraud detection.

The key architectural insight here is the link between DBSCAN and SVM: DBSCAN discovers the gangs (dense, closely related behaviors), and SVM looks for the stragglers among the assassins (lone wolves, outliers). It provides an effective and interpretable pipeline by partitioning the problem into two components.

The results on the Amazon Appliances dataset demonstrate that the pipeline can handle large volumes of review data and identify meaningful behavioral structure. The relative rarity of the high-lift itemsets suggests that this dataset is likely quite clean, although the pipeline provides tools to detect campaigns when present. The absence of strong fraud signals is itself informative: it demonstrates the system’s controlled sensitivity and validates its calibration for operational deployment as a screening tool.

Future work would parallelize the frequent-itemset stage using distributed FP-Growth, add graph features, such as PageRank, to the reviewer-product bipartite graph, and test on other Amazon product categories or datasets from different platforms to see whether the same patterns hold across review ecosystems.

References

- [1] J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann, 2022.
- [2] J. Leskovec, A. Rajaraman, and J. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge University Press, 2020.
- [3] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules in large databases,” in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, pp. 487–499, 1994.
- [4] A. Z. Broder et al., “Syntactic clustering of the web,” *Computer Networks*, vol. 29, no. 8, pp. 1157–1166, 1997.
- [5] N. Jindal and B. Liu, “Opinion spam and analysis,” in *Proc. WSDM*, pp. 219–230, 2008.
- [6] M. Ott et al., “Finding deceptive opinion spam by any stretch of the imagination,” in *Proc. ACL*, pp. 309–319, 2011.
- [7] A. Mukherjee et al., “What yelp fake review filter might be doing?” in *Proc. ICWSM*, pp. 409–418, 2012.
- [8] J. Li et al., “Towards a general rule for identifying deceptive opinion spam,” in *Proc. ACL*, pp. 1566–1576, 2017.
- [9] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. NeurIPS*, pp. 1025–1035, 2017.
- [10] B. Hooi et al., “FRAUDAR: Bounding graph fraud in the face of camouflage,” in *Proc. KDD*, pp. 895–904, 2016.